

Build Aware Knowledge Base Ingestion Guide

The **Build Aware Knowledge Base** provides fast, offline-capable, in-browser full-text search (FTS) across technical documentation, engineering specifications, vendor cut-sheets, NEC code books, safety manuals, and standard operating procedures.

It utilizes an embedded **SQLite FTS5 engine** with **Porter Stemming**, **BM25 relevance ranking**, and **Mozilla PDF.js multi-page rendering** with seamless tab reuse and deep-linking.

1. Directory Structure & Organization

You can organize your files in the `faq/` directory or any of the recognized system directories:

```
JAMES_Development/
├── faq/                                # Primary folder for drop-in FAQs & guides
│   ├── safety/                        # Auto-categorized as 'safety'
│   │   ├── QSG-Arc Flash Labeling.pdf
│   │   ├── nec_code/                # Auto-categorized as 'nec_code'
│   │   │   ├── NEC_Article_240_Overcurrent.md
│   │   ├── equipment/              # Auto-categorized as 'equipment'
│   │   │   ├── Eaton_Pow_R_Line_Specs.pdf
│   │   ├── general/                # Auto-categorized as 'general'
│   │   │   └── Field_Survey_FAQ.md
├── docs/                             # Engineering PRDs, Specs, and CONOPS
├── data/
│   ├── faq/                         # Alternative data FAQ storage
│   ├── knowledge_base/              # Raw document cache
│   ├── knowledge_base.db            # SQLite FTS5 database (auto-generated)
├── tools/
│   └── ingest_kb.py                 # CLI ingestion script
```

2. Supported File Formats & Chunking

Format	Extension	Chunking Method	Special Features
PDF Documents	.pdf	Page-by-page & paragraph segmentation	Captures exact page numbers for automatic viewer jumping
Markdown	.md, .markdown	Heading segmentation (#, ##, ###)	Extracts document titles, sections, and #hashtags
HTML Guides	.html, .htm	Clean DOM text extraction	Preserves semantic headings (h1–h4)
Plaintext / Logs	.txt, .json, .log	Paragraph & block segmentation	Strips noise while preserving technical terms

3. Auto-Category Inference

When scanning files, the ingester automatically determines the category based on: 1. **Subfolder Name:** Files located in `faq//...` inherit the subfolder name (e.g. `faq/safety/` &arr; `safety`, `faq/nec_code/` &arr; `nec_code`). 2. **Filename Keywords:**

- `safety`, `arc` &arr; `safety`
 - `prd`, `spec`, `layout` &arr; `specs`
 - `conops`, `guide`, `workflow` &arr; `workflow`
 - `nec`, `code`, `rule` &arr; `nec_code`
 - `breaker`, `panel`, `transformer`, `switch`, `object` &arr; `objects`
 - `catalog`, `vendor` &arr; `catalog`
 - `faq` &arr; `faq`
-

4. CLI Ingestion Commands

Run all ingestion commands from the project root (`JAMES_Development/`):

A. Full Default Scan (Recommended)

Scans `faq/`, `data/faq/`, `docs/`, `data/knowledge_base/`, and workspace PDFs recursively:

```
python -m tools.ingest_kb
```

B. Ingest a Specific Folder Recursively

```
# Scan a custom folder and all subfolders
python -m tools.ingest_kb --path faq/
```

```
# Scan with a forced category override
python -m tools.ingest_kb --path path/to/manuals/ --category manuals
```

C. Ingest a Single File

```
python -m tools.ingest_kb --file "faq/safety/QSG-Arc Flash Labeling.pdf" --category safety
```

D. Clear & Re-index Knowledge Base

```
# Wipes existing index and rebuilds from scratch
python -m tools.ingest_kb --clear
python -m tools.ingest_kb
```

E. View Knowledge Base Statistics

```
python -m tools.ingest_kb --stats
```

Output Example:

```
=====
Build Aware Knowledge Base Statistics
=====
Total Chunks Indexed: 453
Distinct Files:      68
Categories:          safety, specs, workflow, nec_code, objects, catalog, general

Files Indexed:
  • QSG-Arc Flash Labeling.pdf  [PDF ] [SAFETY  ] 12 chunks
  • PANEL_RULES_AND_LAYOUT_SPEC.md [MD ] [SPECS  ] 14 chunks
=====
```

F. Test Search from CLI

```
python -m tools.ingest_kb --search "arc flash boundary 1.2 cal"
```

5. In-Browser Document Viewer & Tab Reuse

The knowledge base search in the Survey App and Field Data Collector interacts with documents as follows:

1. Pure In-Browser Rendering:

- PDFs render using **Mozilla PDF.js** on canvas at route GET /kb/viewer/{filename:path}.
- Prevents external desktop applications (like Adobe Acrobat) from hijacking the document.

2. Smart Single-Tab Reuse:

- Each distinct document opens in a dedicated, named browser tab (kb_doc_).
 - Clicking different page matches from the search results (e.g. Page 4 & Page 8) automatically communicates via BroadcastChannel('build_aware_kb_channel') to smoothly scroll to that page in the existing tab without spawning duplicate tabs or reloading the file.
-

6. HTTP API Reference

The backend exposes these REST endpoints:

- GET /api/kb/search?q={query}&category={category}&limit={limit}: Execute FTS5 BM25 search with highlighted snippets.
- GET /api/kb/stats: Retrieve chunk counts, file lists, and category breakdowns.
- GET /api/kb/view/{filename:path}: Stream raw document files inline.
- GET /kb/viewer/{filename:path}?page={N}&title={title}§ion={section}: Dedicated HTML5 multi-page document viewer.
- POST /api/kb/ingest: Upload and index a document via multipart form upload.